Nesne Yönelimli Programlama – Konu Anlatımı

Yaşam Döngüsü Modelleri ve Stable Intermediate Forms

Slaytlar, ders kitabı (Schach 8e), ders videoları ve WhatsApp grubu temel alınmıştır

9 Kasım 2026

2 Yaşam Döngüsü Modelleri (Chapter 2)

Bu bölümün sınavdaki yeri

Chapter 2'nin özü şudur: *gerçek hayatta yazılım doğrusal geliştirilmez; hatalar yapılır ve gereksinimler değişir. Bunun etkisini azaltmak için yinelemeli ve artırımlı (iterative and incremental) model ile sağlam (robust) bir mimari gerekir.* Son iki yılın vizelerinde bu bölümle ilişkili **Stable Intermediate Forms** sorusu (15 puan, açıklamalarıyla) geldi. Ayrıca *moving target problem*, *regression fault*, *Miller's law*, *iteration* ve *incrementation* kısa tanım olarak sorulur.

2.1 Teoride ve pratikte yazılım geliştirme

İdeal olarak yazılım Chapter 1'deki gibi **doğrusal (linear)** ve **sıfırdan (from scratch)** geliştirilir: requirements → analysis → design → implementation. Gerçek dünyada ise bu tamamen farklıdır, çünkü:

1. İnsanlar **hata yapar** (we make mistakes).
2. Ürün geliştirilirken **müşterinin gereksinimleri değişir** (moving target problem).

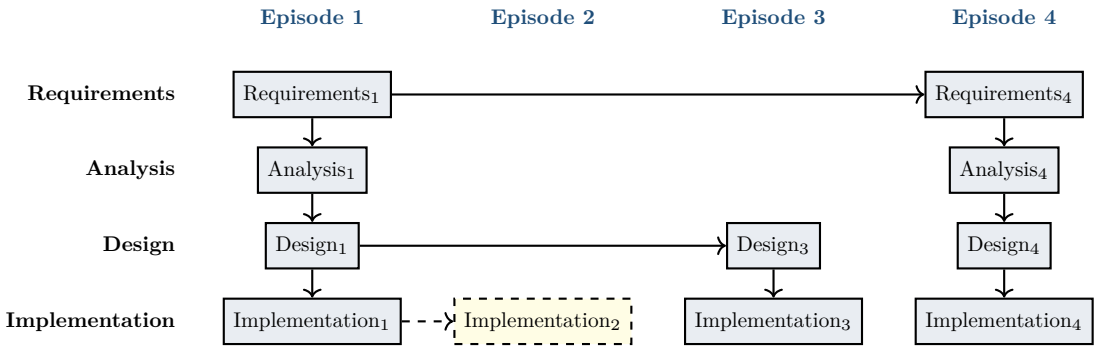
2.2 Winburg mini vaka çalışması

Winburg (Indiana) belediyesi trafiği azaltmak için otobüs sistemi kurar. Her otobüste yalnızca 1 dolarlık banknot kabul eden bir **ücret makinesi** vardır; yazılım bir görüntü tanıma algoritmasıyla banknotun geçerli olup olmadığına karar verir. Gereksinimler: ortalama yanıt süresi **1 saniyenin altında**, doğruluk **en az %98**.

Bölüm	Ne oldu?	Ne tür değişiklik?
Episode 1	Yazılımın ilk sürümü gerçekleştirildi.	–
Episode 2	Yanıt süresi 10 saniye çıktı. Sebep: %98 doğruluk için bütün hesaplarda *double precision* kullanılmış. Single precision'ın da yeterli olduğu anlaşıldı, kodda düzeltmeye başlandı.	**Implementation fault** → corrective
Episode 3	Değişiklik bitse bile süre 4,5 s olacaktı; sorun karmaşık algoritmaydı. Yeni, hızlı bir algoritmayla **yeniden tasarlandı** ve gerçekleştirildi.	**Design fault** → corrective
Episode 4	Belediye başkanı ürünü satmak için doğruluğun artırılmasını istedi. Yeni tasarımla doğruluk %99,5'e çıktı.	**Gereksinim değişikliği** → perfective
Epilogue	Birkaç yıl sonra sensörler eskidi, donanım değişti; yazılım yeniden yazıldı ve aynı sorunlar tekrar yaşandı.	Adaptive

2.2.1 Evolution-tree (evrim ağacı) modeli

Waterfall modeli geri besleme oklarıyla Winburg'u gösterebilir ama **olayların sırasını gösteremez**. Evolution-tree modeli her bölümde hangi artifact'ın hangi sürümünün üretildiğini ve olayların sırasını açıkça gösterir.



Kesikli kutu (Implementation_2) tamamlanmayan işi gösterir. Her bölümün sonunda bir **baseline** (o anki artifact'ların tam kümesi) vardır. Örnek: Episode 3 sonundaki baseline = $\textit{Requirements}_1$, $\textit{Analysis}_1$, $\textit{Design}_3$, $\textit{Implementation}_3$.

2.2.2 Winburg'un dersleri

Gerçek dünyada geliştirme Winburg'dan da kaotiktir. **Değişiklik her zaman gerekir**: yazılım sürekli değişen gerçek dünyanın modelidir ve yazılımcılar insandır, hata yapar.

2.3 Teal Tractors mini vaka çalışması ve moving target problem

Teal Tractors yazılımı yapılırken şirket **Kanada'ya genişler**. Gereken değişiklikler: yeni satış bölgeleri eklenmeli; Kanada vergileri ve farklı iş kuralları desteklenmeli; iki para birimi (USD ve CAD) desteklenmeli. Bu değişiklikler şirket için harika, yazılım için felaket olabilir.

- **Moving target problem:** *A change in the requirements while the software product is being developed.* Sebep iyi olsa bile ürün olumsuz etkilenir: modüller arasında bağımlılıklar (dependencies) oluşur.
- Ürüne yapılan her değişiklik potansiyel olarak bir **regression fault**'a (görünüşte ilgisiz bir bölümde ortaya çıkan hataya) yol açabilir.

- Çok fazla değişiklik olursa ürünün tamamı yeniden tasarlanıp yeniden gerçekleştirilmek zorunda kalınabilir.
- Büyüyen şirketler her zaman değişir; değişikliği isteyen kişinin yeterli gücü (clout) varsa yapılacak bir şey yoktur. **Moving target probleminin bir çözümü yoktur**; sadece etkisi azaltılabilir.

2.4 Yineleme ve artırım (Iteration and Incrementation)

2.4.1 Miller yasası ve stepwise refinement

Miller's Law: İnsan aynı anda yaklaşık **yedi parça (chunk) bilgiye** odaklanabilir. Daha fazla bilgiyle başa çıkmak için **stepwise refinement** kullanılır:

- O an en önemli olan konulara odaklan;
- Şu an daha az kritik olanları ertele;
- Sonunda her konu ele alınır, ama **önem sırasına göre**.

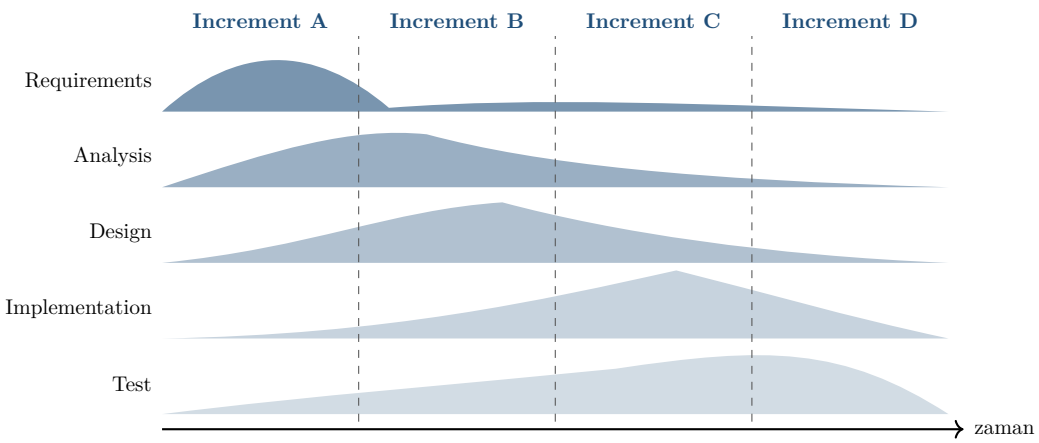
Bu, **artırımlı (incremental)** bir süreçtir. Hoca bunu *snowball effect* (kartopu etkisi) diye anlatır: önce çekirdek (en öncelikli) kısım yapılır, sonra her artırımda yeni özellikler eklenerek ürün büyür.

2.4.2 Iteration ve incrementation

- **Iteration (yineleme):** Temel geliştirme süreci yinelemelidir; *her ardışık sürüm hedefe bir öncekinden daha yakındır.* Bir artırım içinde, hedefe ulaşılana kadar aynı iş akışları tekrarlanır.
- **Incrementation (artırım):** Ürün parçalara (artırımlara) bölünür; her artırımda ürüne yeni işlevler eklenir.
- İkisi **birlikte** kullanılır: tek bir "requirements phase" veya "design phase" yoktur; her fazın birden çok örneği vardır. Artırım sayısı dört olmak zorunda değildir; yineleme sayısı da üç olmak zorunda değildir.

2.4.3 Klasik fazlar ve iş akışları (workflows)

Gerçek dünyada ardışık fazlar yoktur. Bunun yerine **beş çekirdek iş akışı** bütün yaşam döngüsü boyunca yürütülür: **requirements, analysis, design, implementation, test workflow**. Her an bir iş akışı baskındır: başta requirements, sonda implementation ve test. Planlama ve dokümantasyon bütün yaşam döngüsü boyunca yapılır.



Şekilde her iş akışının alanı, o artırımda o iş akışına harcanan emeği gösterir (kitaptaki Figure 2.4'ün sadeleştirilmiş hâli).

2.4.4 Winburg yeniden: artırımlar ve bakım

Evolution-tree modeli yinelemeli-artırımlı modelin üzerine oturtulursa: her bölüm (episode) bir **artırıma** karşılık gelir; her artırım her iş akışını içermek zorunda değildir; Episode 2 ve 3 **corrective**, Episode 4 **perfective** bakımdır.

2.5 Yinelemeli-artırımlı modelin riskleri ve güçlü yanları

Proje bir dizi **mini projeden** (artırımdan) oluşur. Her mini projede artifact'lar genişletilir (incrementation), kontrol edilir (test workflow) ve gerekirse değiştirilir (iteration). **Her yineleme küçük ama tam bir waterfall modelidir.**

Güçlü yanları:

1. Ürünün doğruluğunu kontrol etmek için **birçok fırsat** vardır; her yineleme test iş akışını içerir, hatalar erken bulunur.
2. Mimarinin **sağlamlığı (robustness)** yaşam döngüsünün başında belirlenebilir. *Architecture:* bileşen modüller ve birbirine nasıl oturdukları. *Robustness:* dağılmadan genişlemeyi ve değişikliği kaldırabilme özelliği.
3. **Riskler erken azaltılabilir** (mitigate).
4. Baştan itibaren **çalışan bir sürüm** vardır; müşteri ve kullanıcılar deneyerek ne değişmesi gerektiğini görür. Kısmi sürümler teslim edilerek yeni ürüne geçiş yumuşatılır.
5. Modelin işe yaradığına dair **deneysel kanıt** vardır: Standish Group CHAOS raporlarında (1994–2006) başarılı proje oranı artmıştır.

Yönetimi: Yinelemeli-artırımlı model de waterfall kadar disiplinlidir, çünkü **art arda uygulanan waterfall modelidir**.

2.6 Diğer yaşam döngüsü modelleri (kısaca)

Kitapta karşılaştırılan modeller: **code-and-fix** (tasarım/spesifikasyon yok, kodla-düzelt; küçük programlar dışında kötü), **waterfall** (belge güdümlü, geri besleme döngülü doğrusal model), **rapid prototyping** (müşteriye hızlıca prototip gösterip gereksinimi netleştirme), **open-source**, **agile processes** (kısa iterasyonlar, sürekli müşteri teması), **synchronize-and-stabilize** (Microsoft modeli; her gün bileşenler birleştirilip test edilir), **spiral** (her döngüde **risk analizi**; odak risklerdir). Hoca bu modellerin ayrıntısına girmediğini söyledi; isimlerini ve birer cümlelik özlerini bilmek yeterlidir.

2.7 Stable Intermediate Forms (kararlı ara formlar)

! Hoca derste vurguladı 2025 dersi ve WhatsApp

Hoca bu konuyu *Moving Target Problem* slaytı (Slide 2.43) açıkken not defterine şöyle yazdı: `Stable Intermediate Forms: -Three Points of Contact; -Copy before release; -Iterative and Incremental Software Development (stepwise refinement, snowball effect)`. WhatsApp: *"stable intermediate forms için de sorarım demişti"*, *"Stable İntermediate Forms'un açıklamalarını isterim dedi."* 2024–25 ve 2025–26 vizelerinin ikisinde de soruldu (15 puan, açıklamalarıyla).

Fikir: Karmaşık bir sistem, **her adımda kararlı (çalışan, tutarlı) ara formlardan** geçerek gelişirse çok daha hızlı ve güvenli gelişir. Bir değişiklik ters giderse en son kararlı forma geri dönülür; her şey baştan yapılmaz. Bu, moving target problemi ve kaçınılmaz hataların etkisini azaltmanın yoludur. Hoca üç yöntem saydı:

1. Three Points of Contact (üç temas noktası)

Kaya tırmanışı / tepe tırmanma (*climbing a hill*) benzetmesidir: Tırmanıcının dört uzvundan **üçü her zaman sabit** bir tutamaktadır, **yalnızca biri** hareket eder. Böylece bir tutamak kayarsa düşmez, sabit üç noktaya geri döner.

Yazılımda: **Aynı anda yalnızca tek bir şey değiştirilir, diğer her şey sabit tutulur.** Değişiklik test edilip kararlı hâle gelince sıradaki değişikliğe geçilir. Değişiklik olumsuz sonuç verirse geri dönmek kolaydır ve hatanın kaynağı bellidir (tek değişiklik yapıldığı için).

Sınava yazılabilecek açıklama

Three points of contact: like a climber who keeps three limbs fixed and moves only one, we change only one thing at a time and keep everything else stable. After the change is tested and becomes stable we change the next thing. If the change fails, we can easily go back to the last stable form and we know which change caused the problem.

2. Copy Before Release (yayından önce kopyala)

Yeni bir sürüm yayımlanmadan / değişiklik canlıya alınmadan önce **mevcut kararlı sürümün kopyası (yedeği) alınır**. Yeni sürümde sorun çıkarsa hemen kararlı kopyaya dönülür. Sürüm kontrolü (version control) ve *baseline* kavramı bunun uygulamasıdır.

Sınava yazılabilecek açıklama

Copy before release: before releasing a new version, a copy (backup) of the current stable version is taken. If the new release causes problems, we can roll back to the stable copy. Each copy is a stable intermediate form (baseline).

3. Iterative and Incremental Software Development

Yazılım tek seferde değil, **tekrarlanan döngüler (iteration)** ve **küçük parça eklemeleri (incrementation)** ile geliştirilir. Her artırımın sonunda *çalışan ama eksik* bir ürün vardır; bu her biri kararlı bir ara formdur. Önce en önemli çekirdek yapılır (**stepwise refinement**, Miller yasası), sonra ürün kartopu gibi büyür (**snowball effect**). Riskler azalır, yeni bilgiyle evrimsel gelişim sağlanır.

Sınava yazılabilecek açıklama

Iterative and incremental development: the product is divided into increments (mini projects). In each increment the workflows are iterated until the target is reached, so at the end of every increment we have a working but incomplete product, i.e. a stable intermediate form. We start with the most important core (stepwise refinement, Miller's law) and grow the product step by step like a snowball (snowball effect).

Sık yapılan hatalar

- Sadece başlıkları yazmak yeterli değil; hoca **açıklamalarını** istiyor.
- "Stable intermediate *terms*" değil **forms**. Bazı öğrenci notlarında "terms" yazıyor; doğrusu *forms*.

- WhatsApp'ta Türkçe cevabın geçen yıl kabul edilmediği söyleniyor. Açıklamaları İngilizce yazmaya çalışın; zorlandığınız yerde kısa Türkçe ekleme yapılabilir (hoca: "zorlandığınız kısımda azıcık yazabilirsiniz").

2.8 Kısa tanım soruları

Moving target problem A change in the requirements while the software product is being developed. It induces dependencies and can cause regression faults. There is no solution; its effects can only be minimized.

Regression fault A fault in an apparently unrelated part of the software caused by a change.

Miller's law At any one time we can concentrate on only about seven chunks of information.

Stepwise refinement Concentrate on the currently most important aspects and postpone the less critical ones; every aspect is eventually handled in order of importance.

Iteration Each successive version is closer to the target than its predecessor.

Incrementation Adding new functionality step by step; the product is built as a set of increments (mini projects).

Baseline A complete set of artifacts at a point in time (e.g. at the end of an episode).

Robustness The property of being able to handle extensions and changes without falling apart.